Analysis of Malleable MPI Workloads

Ajith Krishna Kanduri
akanduri@umass.edu

Abstract

This study focuses on the creation of a test-bed for executing malleable MPI workloads, incorporating three representative applications: Jacobi iteration, the conjugate gradient method, and the N-body problem. The test-bed was used to investigate performance and efficiency metrics, including the relationship between average iteration time and the number of nodes, the speedup achieved with increasing nodes and varying CPU affinities, and the impact of node count and CPU affinity configurations on power consumption. These analyses aim to provide insights into the behavior of malleable MPI applications in distributed computing environments.

1. Motivation

This work was inspired by efforts to replicate the experiments of the Carbon Provisioner, an extension of SpeculativeScheduler3, which integrates carbon-aware scheduling and provisioning policies [1]. While the original study relied on simulations, this project aimed to extend these analyses to real-world job workloads. Although a SLURM environment was successfully set up using Docker containers, the lack of readily available malleable MPI workloads posed a significant challenge, necessitating the creation and customization of test cases to enable experimentation.

2. Related Works

The N-body simulation implemented in this study is inspired by the GAIA project [2], which explores distributed computing solutions for large-scale astrophysical simulations. The Jacobi iteration and conjugate gradient method are standard MPI workloads commonly used for benchmarking parallel systems and evaluating distributed computing environments.

The addition of malleability to MPI workloads has been extensively studied as a means to enhance resource utilization and flexibility in high-performance computing systems. Works such as FLEX MPI[3] and Towards Realizing the Potential of Malleable Jobs [4] demonstrate the advantages of enabling MPI applications to adapt dynamically to varying resource availability. These studies highlight benefits such as reduced job completion time, improved system throughput, and better energy efficiency. Malleability is particularly beneficial in modern systems that face fluctuating workloads and energy constraints, making it a key area of innovation in resource-aware scheduling.

3. Method

Based on previous works, experiments were conducted to analyze:

- The relationship between average iteration time and the number of nodes
- Speedup as a function of the number of nodes
- Speedup under different CPU affinity configurations across varying node counts
- Power consumption trends relative to the number of nodes, with a focus on the impact of different CPU affinity settings

3.1. Jobs

- **NBody**: Inspired by GAIA's implementation, this workload models the gravitational

interactions between bodies, distributing computations across nodes using MPI.

- **Jacobi Iteration**: An iterative algorithm for determining the solutions of a strictly diagonally dominant system of linear equations.
- **Conjugate Gradient Iterative**: An algorithm for the numerical solution of particular systems of linear equations, namely those whose matrix is symmetric and positive-definite.

3.2. Checkpointing and Restoration

To introduce malleability, each workload was enhanced with checkpointing capabilities. These checkpoints periodically save the application's state, allowing restoration and resumption from the saved state. This mechanism supports dynamic adjustments to resources, such as changing the number of nodes during execution, improving flexibility and fault tolerance.

3.3. Environment

The experiments were conducted on a system named obelix23, powered by an Intel Xeon E5-2660 v3 processor with the following specifications:

- 40 CPUs: Each processor core is configured for 2 threads per core
- 2 NUMA nodes: NUMA node 0 contains cores 0, 2, 4, ..., 18, and NUMA node 1 contains cores 1, 3, 5, ..., 19
- CPU frequency: All cores were set to the same frequency using the cpupower frequency-set utility
- ePDU: An ePDU is attached to record power consumption values during the experiments

4. Results

The results and codebase for these experiments can be found in the *MPI Workload Repository* [5].

4.1. Relationship Between Average Iteration Time and Number of Nodes

- The average iteration time decreases as the number of nodes increases for all applications.
- This trend suggests good scalability initially, but the benefits of adding more nodes diminish due to communication overhead.

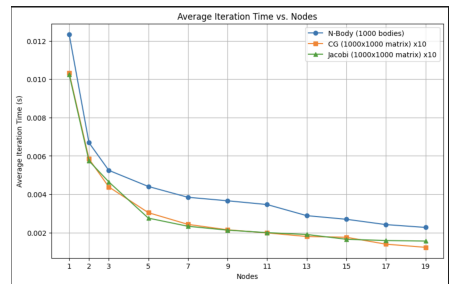

Figure 1: Average Iteration Time vs Number of Nodes

4.2. Speedup as a Function of the Number of Nodes

- Speedup increases with the number of nodes for all applications.
- The Conjugate Gradient (CG) achieves the highest speedup, followed by Jacobi and then N-Body.

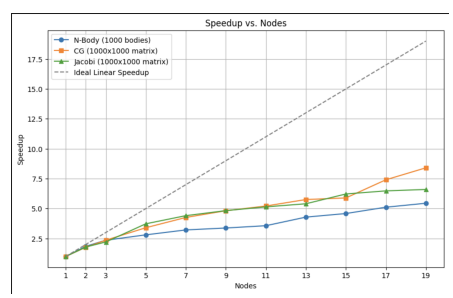

Figure 2: Speedup vs Number of Nodes

4.3. Impact of CPU Affinity Configurations

- The configurations are defined as follows:

- **Even:** Cores are assigned in even indices (0, 2, 4, ..., 38). Cores 0-18 are physical cores, while the rest are hyperthreads.
- **Sequential (Seq):** Cores are assigned sequentially from 0 to 19. All assigned cores are physical.
- **Even Till 18 Then Odd:** Cores 0-18 are assigned in even indices (0, 2, 4, ..., 18), followed by odd indices (1, 3, 5, ...) to ensure no hyperthreads are used.

- The speedup trends are similar across the three CPU affinity configurations.
- On increasing the number of iterations from 10k to 1M, there seems to be less variation in speedups across different configurations.

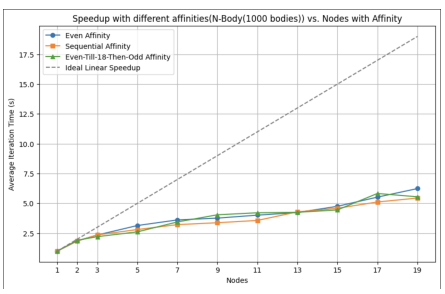

Figure 3: Speedup Across Different CPU Affinity Configurations

4.4. Power Consumption Trends

- The power consumption increases with the number of nodes for all CPU affinity configurations.
- The three affinity configurations—**Even**, **Sequential**, and **Even-Till-18-Then-Odd**—exhibit comparable power trends, with minimal differences across the range of node counts.
- At lower node counts, the power consumption for all configurations is nearly identical. Differences begin to appear as the node count increases, but the variations remain small.

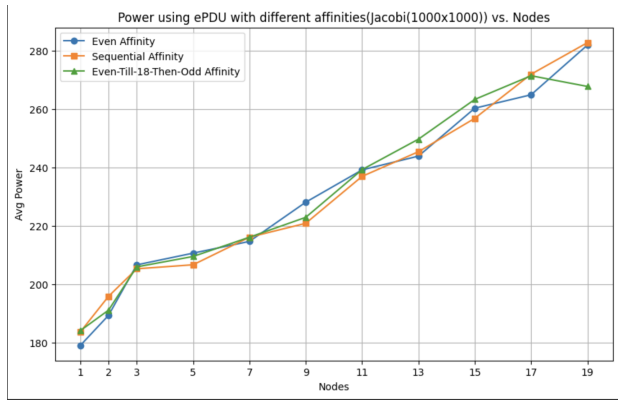

Figure 4: Power using ePDU with different affinities (Jacobi(1000x1000)) vs. Nodes

5. Future Works

- **Scalability Across Multiple Machines:** Extend the current experiments by deploying malleable MPI workloads on a distributed cluster with n different nodes, rather than limiting the setup to a single machine. This would provide insights into the performance and efficiency of these workloads in real-world, large-scale distributed environments.
- **Signal Handling for SLURM Signals:** Enhance the workloads by incorporating robust signal handling mechanisms to respond to SLURM job signals (e.g., `SIGUSR1` and `SIGUSR2`). This would enable dynamic adjustments such as resource deallocation or checkpointing based on scheduler-driven triggers, further improving flexibility and resilience.

6. References

References

[1] Carbon Provisioner Repository: `https://github.com/umassos/carbon-provisioner/tree/motivation-experiment`

[2] GAIA Project Repository: `https://github.com/umassos/GAIA/tree/main/jobs/nbody`

[3] FLEX-MPI: https://www.uc3m.es/ss/Satellite/GruposInvestigacion/en/TextoDosColumnas/1371351717423/?d=Touch

[4] Towards Realizing the Potential of Malleable Jobs: http://charm.cs.illinois.edu/newPapers/14-29/paper.pdf

[5] MPI Workload Repository: https://github.com/umassos/MPI-Workload